UrbanVision AI

AI-powered smart-city complaint management: snap a civic problem, the AI recognises it, routes it to the right municipal department and tracks it until it is fixed.

PROJECT WORK

Jyoti Kumari Verma

URN 2302573 · CRN 2315117 · TR-103

Guru Nanak Dev Engineering College

Source code: **github.com/Jyoti-verma24/Urbanvision-AI**

OVERVIEW

Abstract

Civic problems such as potholes, open manholes and overflowing bins are usually reported through phone calls or paper forms. These reports are slow, often reach the wrong office, are filed many times for the same spot, and the citizen rarely hears back.

UrbanVision AI is a web application that turns a single photo into a complete, trackable complaint. A **YOLOv8s** object-detection model, trained on 2,518 labelled street photos, recognises six kinds of civic issue and draws boxes around them. The system then scores how urgent the complaint is, sends it to the responsible municipal department, merges duplicate reports made within 100 m of each other, escalates complaints that pass their deadline, and emails the citizen a complaint ID to track progress until the problem is fixed.

The project includes a citizen portal in English, Hindi and Punjabi, department-wise staff dashboards, a public open-data page, security protections (hashed passwords, CSRF tokens, login lockout) and an automated test suite of 180 checks.

6	86%	180
civic issue types detected	mAP@50 on potholes (test set)	automated checks, all passing

Contents

1. Problem statement and objectives 3

2. System architecture and technology 4

3. Modules and how a complaint flows 5

4. Smart handling: priority, routing, duplicates, escalation 6

5. AI model: dataset, training and results 7

6. Screenshots of the working system 9

7. Security and testing 11

8. Limitations, future scope and conclusion 12

CHAPTER 1

Problem statement and objectives

The problem

Slow

Phone calls and paper forms take days to reach anyone who can act.

Misrouted

Reports land at the wrong office and wait there until someone forwards them.

Repeated

Ten people report one pothole and the city sees ten separate tickets.

Invisible

Citizens never learn whether anything was fixed, so they stop reporting.

Objectives

- Let any citizen report a problem with just a **photo, a short note and a location**, without creating an account.
- Use a **deep-learning object detector** to recognise the issue type automatically, so the citizen does not have to classify it.
- **Route** every complaint to the correct department and **rank** it by urgency with a transparent priority score.
- **Merge duplicate** reports of the same issue at the same place and **escalate** complaints that pass their deadline.
- Give each department a **dashboard** that shows only its own work, with before/after photos as proof of fix.
- Keep citizens informed by **email** and a **tracking page**, and publish city-wide performance on a **public page**.
- Be usable by everyone: **three languages**, phones, dark mode and reduced-motion support.
- Be **secure** and **tested**: hashed passwords, CSRF protection, login lockout, and an automated test suite.

Scope

The system covers six issue types handled by five departments: potholes and road cracks (Roads), open manholes (Sewerage), overflowing waste containers (Sanitation), faulty traffic lights (Electrical) and stray animals on the road (Animal Control).

CHAPTER 2

System architecture and technology

The whole system runs as one Python (Flask) server. The browser talks to Flask; Flask calls the vision model for the photo and an optional language model for the description, and stores everything in SQLite.

Users
Citizens, department staff, super admin, public visitors

→

Web server
Flask routes, logins, priority score, duplicate check, escalation

→

AI
YOLOv8s for photos; Claude for text, with a rule-based fallback

→

Storage
SQLite database, photos and AI results, email alerts

Technology stack

Layer	Technology
Language	Python 3.11
Web framework	Flask 3.1, Jinja2 templates, Werkzeug (password hashing)
Object detection	Ultralytics YOLOv8s (PyTorch), OpenCV, NumPy
Language understanding	Claude API (optional) with a rule-based fallback
Database	SQLite
Front end	HTML, CSS, JavaScript; Leaflet maps, Chart.js charts, Lucide icons
Notifications & reports	Flask-Mail (email), ReportLab (PDF), openpyxl (Excel)
Training	Google Colab, NVIDIA T4 GPU
Version control	Git, GitHub

Database design

Table	Main fields
complaints	id, issue, description, email, location, lat/lng, department, severity, confidence, priority, status, created/started/resolved times, before and after photos, staff note, rating, escalated flag, number of reports
staff	username, password hash (scrypt), department
activity_log	activity text, department, time (audit trail on the dashboard)

CHAPTER 3

Modules and how a complaint flows

Complaint lifecycle

1 · Snap
Citizen uploads a photo, note, location and email

→

2 · Detect
YOLOv8s boxes and names the issue

→

3 · Route
Priority score; sent to the right department

→

4 · Fix
Pending → In progress → Resolved, with an after photo

→

5 · Close
Citizen is emailed and rates the fix ★1–5

Modules

Citizen portal

- Report with photo, note, map pin or phone GPS
- Instant AI result and complaint ID by email
- Track with complaint ID + email
- Rate the fix once resolved

AI detection

- YOLOv8s finds every issue in the photo in one pass
- Draws boxes with confidence scores
- Confidence sets the severity (Critical ≥ 90%, High ≥ 75%, Medium ≥ 50%)

Language understanding

- Reads the description in English, Hindi or Punjabi
- Finds urgency words and sensitive places
- Adds an English translation for staff
- Works without an API key using rules

Department dashboard

- Each department sees only its own queue
- Map, charts, activity log, PDF and Excel export
- Start work, resolve with an after photo
- Super admin creates staff accounts

Public open data

- Issues reported, percent fixed, days to fix
- Scorecard for each department
- No login and no personal details

Model performance page

- Test-set scores of the model in use
- Accuracy per issue type
- Confusion matrix and PR curves

CHAPTER 4

Smart handling

Priority score (0–100)

Every complaint gets a score that decides its place in the department's work queue:

```
priority = 0.5 × AI confidence              (up to 50)
         + severity   Critical 30 · High 20 · Medium 10 · Low 5
         + urgency    Critical 20 · High 12 · Low 5   ("accident", "children" …)
         + 10 if a sensitive place is named   (school, hospital, market …)
         + 15 once the complaint is overdue   (total capped at 100)
```

Score	Band	Score	Band
80–100	**P1 Urgent**	40–59	P3 Normal
60–79	P2 High	0–39	P4 Low

Department routing

Issue	Department
Pothole, road cracks	Roads
Open manhole	Sewerage
Waste container	Sanitation
Traffic lights	Electrical
Stray animal	Animal Control

Duplicate detection

A new report of the **same issue within 100 m** of an open complaint (haversine distance between GPS points) is counted as a **confirmation** of that complaint instead of a new ticket.

Location is taken, in order, from the map pin, the phone's GPS, the GPS stored in the photo (EXIF), or a known area name.

Escalation

Deadlines by severity: **Critical 2 days, High 5, Medium 10, Low 15**. A complaint past its deadline is flagged **Overdue** and gains +15 priority.

> Because the language step understands Hindi and Punjabi, “स्कूल के बाहर बड़ा गड्ढा” is scored as urgently as “huge pothole outside the school”.

CHAPTER 5

AI model: dataset and training

Dataset

2,013
training images (80%)

252
validation images (10%)

253
test images, never used in training

2,518 street photos labelled with bounding boxes in YOLO format across six classes: **pothole, cracks, open_manhole, waste_container, traffic_lights, animal**. It also contains 100 “good road” photos with no boxes, so the model learns that a normal road is background and not a problem.

Why YOLOv8

YOLO (“You Only Look Once”) is a single-stage detector: it looks at the whole image once and predicts all boxes together, which makes it fast enough to run on upload.

Input
Photo resized to 640 × 640

→

Backbone
CNN extracts edges, shapes and textures

→

Neck
Combines small and large scales

→

Head
Anchor-free: box, class and confidence

→

Filter
NMS removes overlaps; keep ≥ 25%

Training

- **Transfer learning:** started from COCO-pretrained YOLOv8s weights (11.2 million parameters).
- **Hardware:** Google Colab with an NVIDIA T4 GPU (training on the laptop CPU ran out of memory).
- **Epochs:** set to 100 with early stopping; training stopped at epoch 87, with the best validation score at epoch 46.
- **Baseline:** a smaller YOLOv8n trained for 15 epochs on a CPU, kept for comparison.
- **Evaluation:** both models scored by `ai_model/evaluate.py` on the same 253 held-out test images; the report is published to the app's `/model` page.

> **Data quality fix:** while preparing the data, 100 good-road images and one crack image were found missing from the training folder and restored before training.

Results

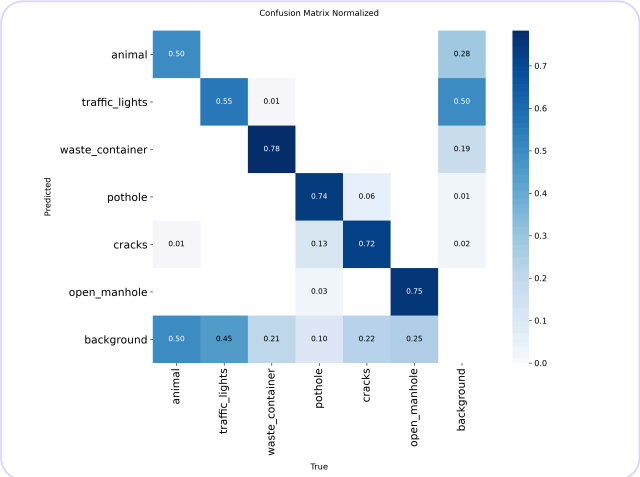
Metric (253 test images)	YOLOv8n (baseline)	YOLOv8s (in use)
Trained on	laptop CPU, 15 epochs	Colab T4, 87 epochs
Precision	0.729	**0.754**
Recall	**0.616**	0.584
mAP@50	**0.647**	0.634
mAP@50–95	**0.379**	0.363
Pothole mAP@50	0.805	**0.860**
Time per photo (CPU)	**83 ms**	165 ms

Why YOLOv8s was chosen: the overall scores are within about one point, but YOLOv8s is better on potholes, the most reported issue, and more precise, so it raises fewer false alarms. On a real street photo it found all four potholes, where YOLOv8n missed the middle one (24% confidence, just under the 25% cut-off).

Accuracy per issue type (YOLOv8s)

Issue	mAP@50	Precision	Recall
Pothole	86.0%	93.4%	72.1%
Waste container	78.7%	79.0%	72.7%
Open manhole	77.8%	89.4%	75.0%
Traffic lights	50.0%	68.5%	45.4%
Road cracks	44.2%	62.1%	44.4%
Stray animal	43.8%	59.9%	40.8%

Finding: a bigger model did not lift the weaker classes, so the project is limited by data, not by model size. Those classes need more labelled photos.



Normalized confusion matrix of YOLOv8s on the test set.

CHAPTER 6

Screenshots: citizen side

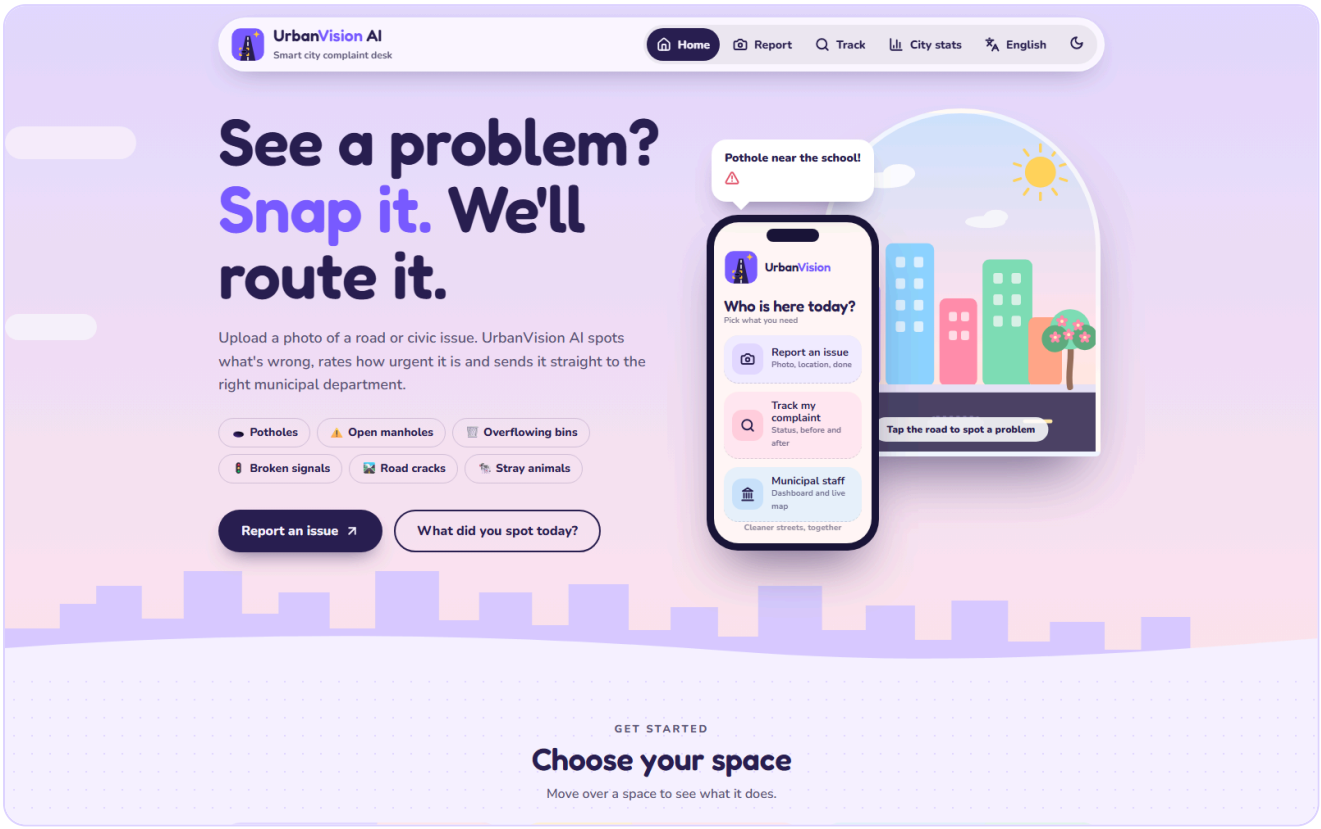


Fig. 1 — Home page: the citizen chooses to report, track or sign in as staff.

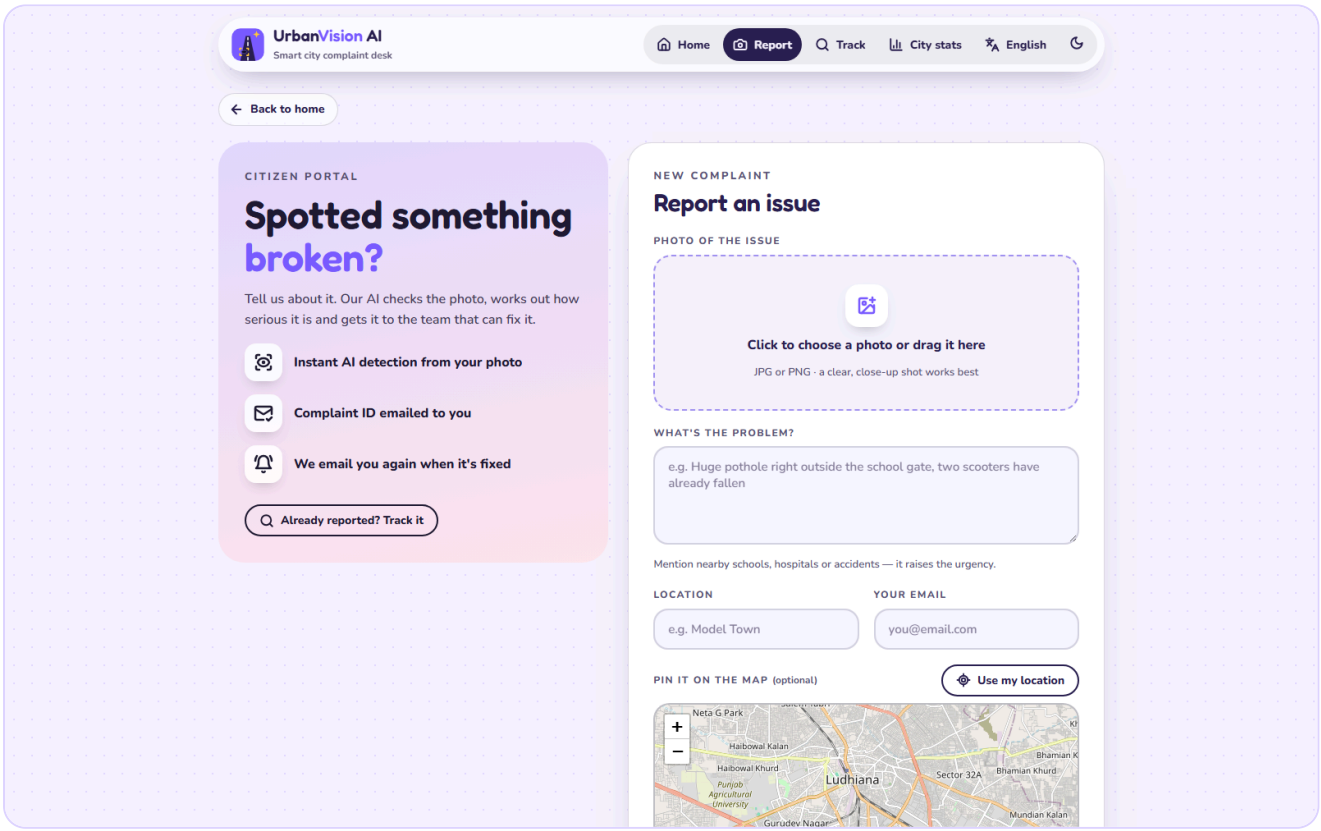


Fig. 2 — Report form: photo, description, location, email and a map pin.

Screenshots: AI result and municipal side

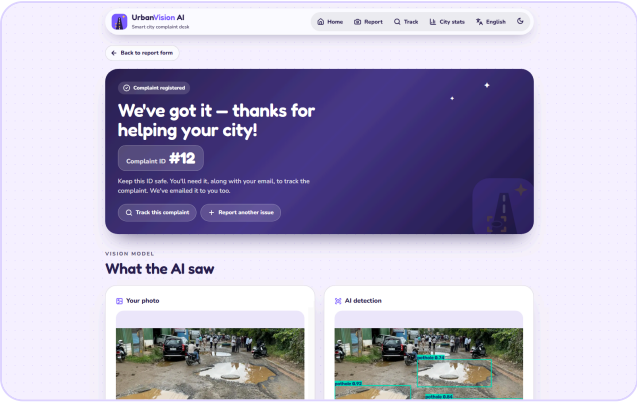


Fig. 3 — Result page with the complaint ID and what the AI saw.



Fig. 4 — Detection output: four potholes, 74–92% confidence.

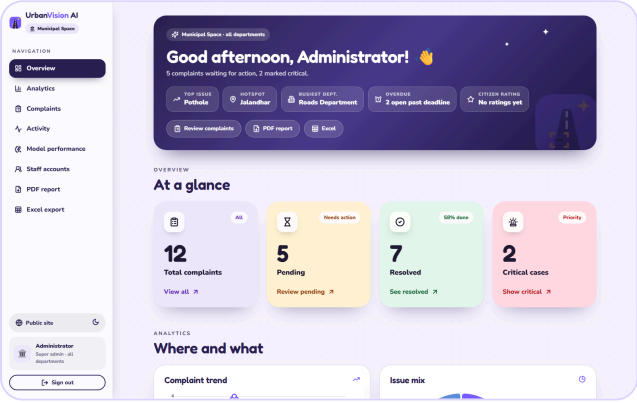


Fig. 5 — Municipal dashboard: pending, resolved, critical and overdue at a glance.

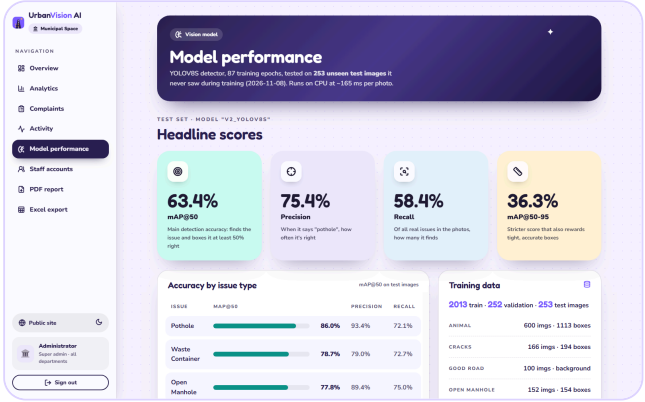


Fig. 6 — Model performance page with live test scores.

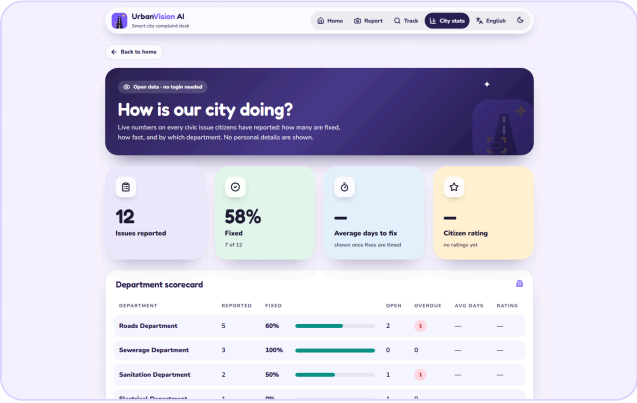


Fig. 7 — Public city stats: open data with no personal details.

About these screenshots

Taken from the running application. The dashboard and public pages show sample test data, so no real citizen details appear.

CHAPTER 7

Security and testing

Security

Risk	Protection
Stolen passwords	Admin and staff passwords are stored as **scrypt hashes**, never in plain text; a script sets a strong admin password.
Cross-site request forgery	A secret **CSRF token** on every form; requests without it are refused.
Password guessing	**Login lockout**: 5 wrong passwords lock the login for 15 minutes.
Session theft	Session cookie is **HttpOnly** and **SameSite=Lax**.
Malicious uploads	Files are opened as images before saving, renamed with a random prefix; non-images are refused.
Seeing others' data	Department-scoped queries and 403 pages; tracking needs complaint ID **and** email; the public page shows no emails or descriptions.

Automated testing

One command runs the whole suite. Each test file runs in a **throwaway copy** of the app with a made-up sample database and a random admin password, so real data is never touched.

```
python tests/run_tests.py          →  180 checks in 11 files, all passing
```

Area tested	Area tested
Uploads and AI detection (several photo types)	Department logins and data scoping
Duplicate merging within 100 m	Public open-data page
Map, GPS and EXIF locations	Hindi and Punjabi translations
Lifecycle and escalation	Language AI (with a fake Claude) and fallback
Security: hashing, CSRF, lockout, cookies	Home page and redirects after forms

CHAPTER 8

Limitations, future scope and conclusion

Limitations

- Cracks, traffic lights and animals score 44–50% mAP because of limited training data.
- About 165 ms per photo on a CPU.
- SQLite suits a demo, not a whole city.
- Escalation runs when the dashboard is opened.

Future scope

- Collect and label more photos of the weaker classes.
- GPU server and PostgreSQL for city-scale use.
- Scheduled escalation and SMS alerts.
- Mobile app and links to city work-order systems.

Conclusion

UrbanVision AI shows that a single photo can replace a slow, manual complaint process. The YOLOv8s model recognises six civic issues and reaches 86% mAP@50 on potholes; the system then scores, routes, de-duplicates and escalates each complaint automatically, while citizens can follow it to the end and the public can see how the city performs. Security protections and 180 automated checks make it a reliable base for a real municipal deployment.

How to run

```
git clone https://github.com/Jyoti-verma24/Urbanvision-AI.git
cd Urbanvision-AI
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
.venv\Scripts\python scripts\set_admin_password.py
.venv\Scripts\python backend\app.py        →  open http://localhost:5000
```

> **Source code:** github.com/Jyoti-verma24/Urbanvision-AI